

DISTRIBUTED AND CLOUD COMPUTING

LAB 10: MAPREDUCE STREAMING & SPARK

MapReduce Streaming

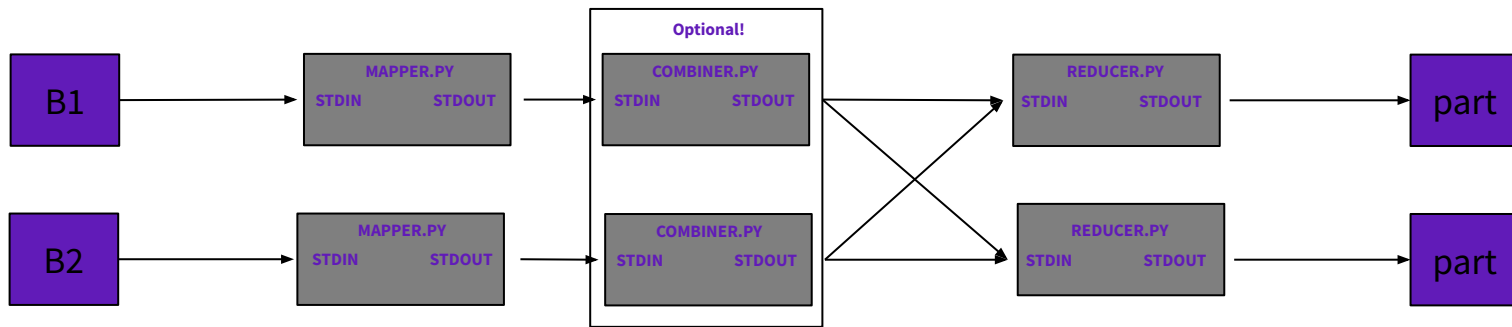
MapReduce Streaming

- So far all the examples of MapReduce we have seen used Java binaries for the mapper and reducer processes
 - Using the pre-compiled examples that shipped with hadoop (pi calculation)
 - Jars compiled by us (word count)
- MapReduce streaming offers the ability to use **ANY** kind of executable for the mapper and the reducer process
 - These can be shell scripts, python scripts etc..

Gives the programmer much more freedom when writing MapReduce programs

MapReduce Streaming

- MapReduce streaming jobs utilise the **stdin** and **stdout** buffers
- Initial data are fed to the **stdin** of the mapper which outputs $\langle k1, v1 \rangle$ to **stdout**
- The **stdout** of mapper is fed to **stdin** of reducer which outputs $\langle k2, v2 \rangle$ to **stdout**
- The **stdout** of the reducer is then written back to HDFS
- By default:
 - New Lines (`\n`) are used to separate the KV pairs
 - The first tab (`\t`) separates the key from the value



Task 2: Mapreduce streaming with Python scripts

Write the **mapper.py** and **reducer.py** python scripts to be used by a MapReduce Streaming jobs that do the following two task on 'Pokemon.csv'

TASK 1: Compute the total number of Pokemons for each pokemon type (only consider primary type 'Type 1')

TASK 2: Compute the total number of Legendary pokemon for each type

TASK 3: Compute the fastest pokemon for:

- Only considering the primary type
- Considering both primary and secondary types

NOTES:

- Use the word count example (wc_mapper, reducer & shell script) as a template to easily run and test your scripts
- The official apache image come with python2 so ensure your python scripts are valid python2 code

Apache Spark

Introduction to Spark



Started at UC Berkeley in 2009 & became an Apache project in 2013

Utilised by a lot of the world's largest organisations to process massive datasets

- NASA relies on Spark to process communication from spacecrafts in the deep space!

SPARK is an in-memory distributed data processing engine

- Written in Scala -> a functional programming language that runs on JVM
- Offers API's in various languages -> we will use Python's API, **PySpark**

Why Spark?

- The key limitation of MapReduce lies in its **reliance on the disk**
- This leads to high latency

Introduction to Spark: RDD's

Spark utilises a Master-Slave architecture for distributed data processing

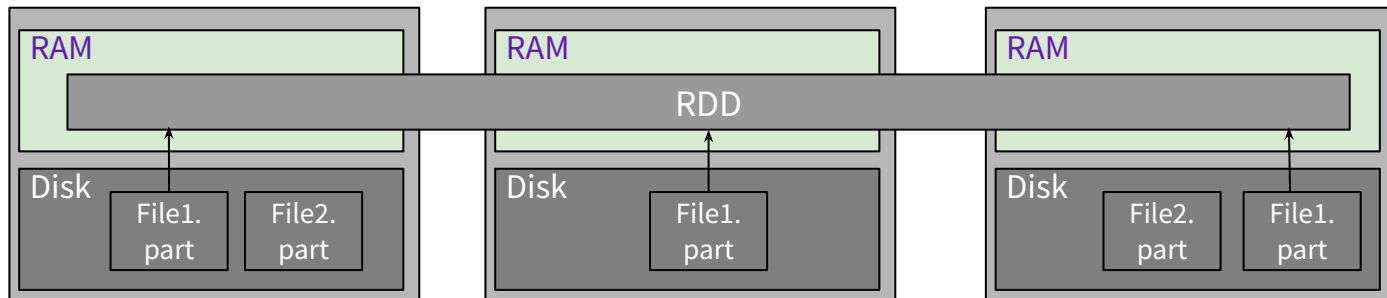
Spark introduces the notion of a **Resilient Distributed Dataset (RDD)**

- **In memory data structure** distributed among the cluster

All data processing in Spark is done using RDDs

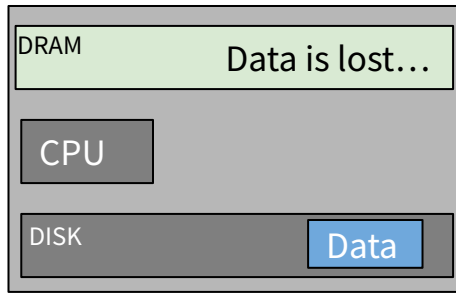
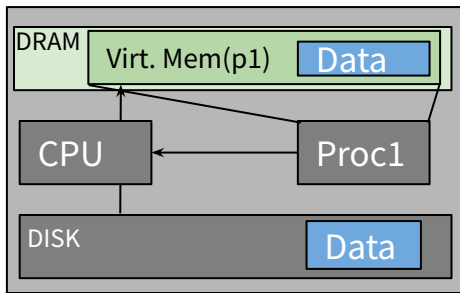
Spark RDDs allow for higher efficiency than MapReduce

- 1) **RDD caching / persistent RDDs**
- 2) **Lazy execution**

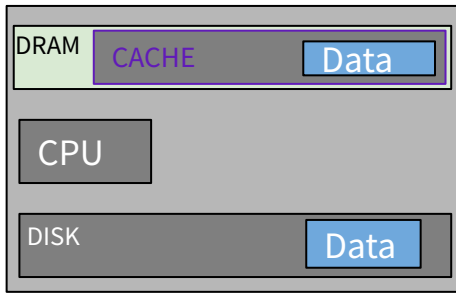
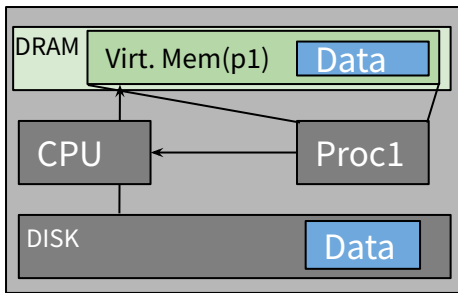
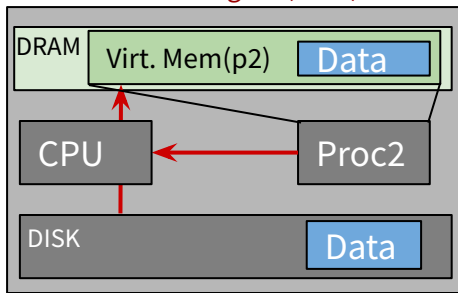


Introduction to Spark: Persistent RDDs

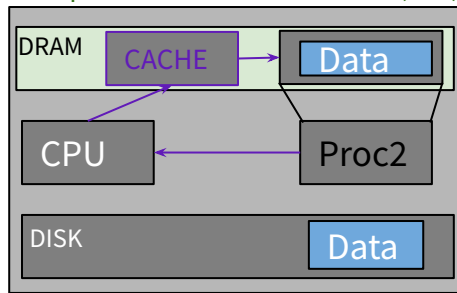
No in-memory cache



Load from disk again (slow)



RDD persistent RDD from cache (fast)



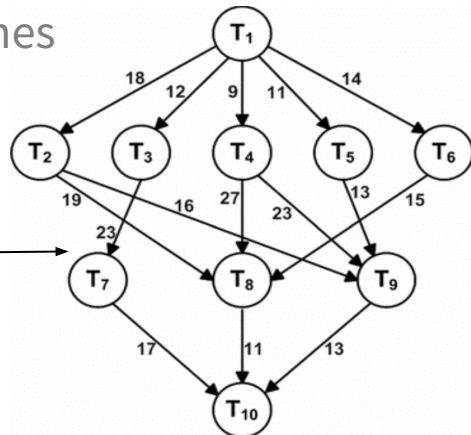
With in-memory cache

Introduction to Spark: Laziness

Spark is **LAZY**: Some expressions are **not** evaluated when executed

Transformations (LAZILY EVALUATED): create a new RDD/DataFrames

- They **DO NOT** execute their logic immediately
- Instead:
 - Reserve memory for produced RDD
 - Add the computation as a node in a **DAG** workload
- **Examples:** map, filter, select, withColumn, join, groupBy



Actions:

- Execute the DAG workload consisting of all unevaluated transformations
- Examples: collect, count, take, show, save / write

Introduction to Spark: Benefits of laziness

Delays execution so it can merge operations and skip redundant work

Pushes filters and projections closer to the source to reduce data touched

- Spark tries to eliminate unnecessary columns and rows as early as possible.
- Shrinks the volume of data flowing through the pipeline

Spark analyzes the full DAG to decide where it must distribute data across the cluster and where it can avoid it.

- Utilises the added context of knowing the entire process before execution

Introduction to Spark: Getting Data

Spark does NOT come with a default file system or any specification for accessing data

How data makes it to the nodes is flexible and up to the user:

- Client can upload the data along with the job
- Nodes can download the data from a database or the cloud

Common approach: Run Spark in an HDFS cluster

- Data is already stored in a distributed manner by the nodes

Introduction to Spark: Spark's components

Spark Core

- Contains the definition of RDD's as well as the transformations and actions

Spark SQL

- Defines a DataFrame structure based on an RDD
- This allows the use of SQL

Task 1: Installing Spark (PySpark)

Installing PySpark is very simple

- Just like installing any other python library `$ pip install pyspark`

It is recommended to utilise a virtual environment like:

- Conda or
- UV (recommended)
 - Init a project `$ uv init`
 - Add pyspark as a dependency `$ uv add pyspark`

It is very useful to use a python notebook for pyspark so consider adding jupyter

- For UV `$ uv add jupyter`

Test your installation by running the spark_intro.ipynb notebook from BB

Task 2: PySpark for distributed data processing

TASK DESCRIPTION: Implement the MapReduce Streaming tasks from before with PySpark SQL

TASK 1: Compute the total number of Pokemons for each pokemon type (only consider primary type 'Type 1')

TASK 2: Compute the total number of Legendary pokemon for each type

TASK 3: Compute the names of the fastest pokemon for:

- Only considering the primary type
- Considering both primary and secondary types
- HINT: check out F.max_by()

NOTE: PySpark SQL is very close to SQL - use the following references to see what is available

- DataFrame operations:
 - <https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/dataframe.html>
- Functions contains useful functions you can use in your transformations
 - <https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/functions.html>